

Wheelzy Technical Assessment — Answers

Candidate: Orieb

Question 1 — Database design, SQL query, Entity Framework

A customer wants to sell a car. We store the car catalog once (make → model → submodel), the car's year and zip on the sale case, buyers who cover zips with a standard quote, one current quote per case, and a status history.

Why the tables are split this way

- **CarMake / CarModel / CarSubmodel** keep Honda, Civic, and EX as one row each. Many cases can point at the same submodel instead of repeating strings.
- **Year stays on SaleCase.** Year belongs to this specific vehicle, not to the catalog.
- **ZipCode** is its own table so buyers and cases share the same location list.
- **Buyer.QuoteAmount** is the standard offer (Buyer ABC covers 10 zips and pays \$500 for each car). **BuyerZipCode** is the coverage list.
- **CaseQuote** copies the amount onto the case. If the buyer later changes their rate, old cases keep the quoted number. **IsCurrent** marks the chosen quote. A filtered unique index guarantees only one current quote per case. Current is not required to be the highest amount.
- **CaseStatusType** includes **RequiresStatusDate**. **Picked Up** has that flag set. A trigger rejects a Picked Up row with a null **StatusDate**. Other statuses may leave **StatusDate** null.
- **CaseStatusHistory** stores every change (who, when) plus **IsCurrent** for the live status.

Schema: `src/Wheelzy.Assessment/Sql/01-schema.sql`

SQL query

Only the requested columns: car year/make/model/submodel, current buyer name and quote, current status name and status date.

```
SELECT
    c.Year,
    mk.Name AS Make,
    md.Name AS Model,
    sm.Name AS Submodel,
    b.Name AS CurrentBuyerName,
    q.Amount AS CurrentQuoteAmount,
    st.Name AS CurrentStatusName,
    h.StatusDate AS CurrentStatusDate
FROM dbo.SaleCase AS c
INNER JOIN dbo.CarSubmodel AS sm ON sm.SubmodelId = c.SubmodelId
INNER JOIN dbo.CarModel AS md ON md.ModelId = sm.ModelId
```

```

INNER JOIN dbo.CarMake AS mk ON mk.MakeId = md.MakeId
LEFT JOIN dbo.CaseQuote AS q
    ON q.CaseId = c.CaseId
    AND q.IsCurrent = 1
LEFT JOIN dbo.Buyer AS b ON b.BuyerId = q.BuyerId
LEFT JOIN dbo.CaseStatusHistory AS h
    ON h.CaseId = c.CaseId
    AND h.IsCurrent = 1
LEFT JOIN dbo.CaseStatusType AS st ON st.StatusTypeId = h.StatusTypeId;

```

Left joins are used so a new case still appears if a current quote or status has not been set yet.

Entity Framework

The same shape, projected to CaseSummaryDto so EF does not load full graphs:

```

return dbContext.Cases
    .AsNoTracking()
    .Select(c => new CaseSummaryDto
    {
        Year = c.Year,
        Make = c.Submodel.Model.Make.Name,
        Model = c.Submodel.Model.Name,
        Submodel = c.Submodel.Name,
        CurrentBuyerName = c.Quotes.Where(q => q.IsCurrent).Select(q =>
q.Buyer.Name).FirstOrDefault(),
        CurrentQuoteAmount = c.Quotes.Where(q => q.IsCurrent).Select(q =>
(decimal?)q.Amount).FirstOrDefault(),
        CurrentStatusName = c.StatusHistory.Where(s => s.IsCurrent).Select(s =>
s.StatusType.Name).FirstOrDefault(),
        CurrentStatusDate = c.StatusHistory.Where(s => s.IsCurrent).Select(s =>
s.StatusDate).FirstOrDefault()
    })
    .ToListAsync(cancellationToken);

```

Code: src/Wheelzy.Assessment/Question1/CaseQueryService.cs

Question 2 — Data that rarely changes but is read all the time

Cache it. Good examples in this domain: makes/models/submodels, zip coverage, status types, buyer standard quotes.

Single instance: IMemoryCache (or a static lookup loaded at startup) is enough. Use a sliding or long absolute expiration, and invalidate when an admin updates the lookup.

More than one instance: yes, it matters.

- In-memory cache is per process. Instance A can hold a stale make list after Instance B updates SQL Server.
- For multiple instances keep a short in-memory cache in front of a **distributed cache** (Redis / IDistributedCache). Writes update the database, then remove or overwrite the Redis key. Optionally publish an invalidation message so other nodes drop their local copy.
- If the data is tiny and almost never changes, loading it once at startup and recycling the app (or pushing a refresh signal) is also valid.

Reference helper: `src/Wheelzy.Assessment/Question2/LookupCache.cs`

Question 3 — UpdateCustomersBalanceByInvoices

Original method problems:

1. `SingleOrDefault(invoice.CustomerId.Value)` is not a predicate, so it does not compile. It should be `c => c.Id == invoice.CustomerId`.
2. One query per invoice (N+1).
3. `SaveChanges()` inside the loop = one transaction per invoice.
4. Missing customer becomes a `NullReferenceException`.
5. `CustomerId.Value` throws if `CustomerId` is null.
6. Two invoices for the same customer should sum, not apply one at a time with extra round-trips.

Rewritten approach: ignore null/empty invoice lists and invoices with no customer; group totals by customer id; load those customers in one query; subtract the grouped total; call `SaveChanges` once.

```
public async Task UpdateCustomersBalanceByInvoicesAsync(
    IReadOnlyCollection<Invoice> invoices,
    CancellationToken cancellationToken = default)
{
    ArgumentNullException.ThrowIfNull(invoices);
    if (invoices.Count == 0) return;

    var totalsByCustomer = invoices
        .Where(invoice => invoice.CustomerId.HasValue)
        .GroupBy(invoice => invoice.CustomerId!.Value)
        .ToDictionary(group => group.Key, group => group.Sum(invoice => invoice.Total));

    if (totalsByCustomer.Count == 0) return;

    var customerIds = totalsByCustomer.Keys.ToList();
    var customers = await dbContext.Customers
        .Where(customer => customerIds.Contains(customer.Id))
        .ToListAsync(cancellationToken);

    foreach (var customer in customers)
        customer.Balance -= totalsByCustomer[customer.Id];

    await dbContext.SaveChangesAsync(cancellationToken);
}
```

Question 4 — GetOrders with optional filters

Keep one `IQueryable<Order>` and add a `Where` only when that filter is set. Null or empty id lists mean “do not filter by that field.”

```
public async Task<List<OrderDTO>> GetOrders(
    DateTime? dateFrom, DateTime? dateTo,
    List<int> customerIds, List<int> statusIds, bool? isActive)
{
    IQueryable<Order> query = dbContext.Orders.AsNoTracking();

    if (dateFrom.HasValue)
        query = query.Where(o => o.OrderDate >= dateFrom.Value);
    if (dateTo.HasValue)
        query = query.Where(o => o.OrderDate <= dateTo.Value);
    if (customerIds is { Count: > 0 })
        query = query.Where(o => customerIds.Contains(o.CustomerId));
    if (statusIds is { Count: > 0 })
        query = query.Where(o => statusIds.Contains(o.StatusId));
    if (isActive.HasValue)
        query = query.Where(o => o.IsActive == isActive.Value);

    return await query
        .Select(o => new OrderDTO
        {
            Id = o.Id,
            Date = o.OrderDate,
            CustomerId = o.CustomerId,
            StatusId = o.StatusId,
            IsActive = o.IsActive,
            Total = o.Total
        })
        .ToListAsync();
}
```

Unused filters never appear in the SQL. `AsNoTracking` plus a DTO projection avoids tracking and extra columns.

Question 5 — Process .cs files

`CSharpFileProcessor` walks a folder (and subfolders), skips `bin / obj / .git`, and uses Roslyn so renames hit real identifiers instead of random text.

Action	Behavior
a	Async methods whose name does not end with <code>Async</code> are renamed. Call sites are left unchanged, as requested.
b	Identifiers ending in <code>Vm</code> , <code>Vms</code> , <code>Dto</code> , <code>Dtos</code> become <code>VM</code> , <code>VMs</code> , <code>DTO</code> , <code>DTOs</code> .

c	Consecutive methods in the same type get a blank line between them if one is missing.
---	---

Tests in `tests/Wheelzy.Assessment.Tests/CSharpFileProcessorTests.cs` cover async rename without updating references, DTO/VM suffix rewrites, inserting a missing blank line, leaving an existing blank line alone, and processing nested folders with all three actions.